

Experiment 1

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using `fork()`. 3. Execute the command in the child process using an appropriate `exec()` system call. 4. Allow the parent process to wait for the child using `wait ()`. 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (`uname`, `lscpu`, `lsblk`, `ps`, `top`), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
    char command[100];
```

```
    printf("Enter a Linux command: ");
```

```
    scanf("%s", command);
```

```
    pid_t pid = fork();
```

```
if (pid < 0)
{
    perror("fork failed");
    return 1;
}
else if (pid == 0)
{
    printf("Child Process\n");
    printf("Child PID: %d\n", getpid());
    printf("Parent PID: %d\n", getppid());

    execlp(command, command, NULL);

    perror("exec failed");
    exit(1);
}
else
{
    printf("Parent Process\n");
    printf("Parent PID: %d\n", getpid());
    printf("Child PID: %d\n", pid);

    wait(NULL);
}
```

```

        printf("Child process completed.\n");
    }

    return 0;
}

```

Output:

```

ashwithreddy_earla@ASHWITHREDDYPC:~$ ./exp1
Enter a Linux command: ls
Parent Process
Parent PID: 612
Child PID: 615
Child Process
Child PID: 615
Parent PID: 612
HELLO      exec          exp1.program4    experiment       experiment5.c    gcc             output.txt      sample.txt
OSSP       exec.c          exp1.program5    experiment2      experiment6.c    input           producure       skill
OSSP.c     expl           exp1.program5.c  experiment2.c   experiment7.c   input.c         producure.c     skill.c
close      expl.c          exp1.program6    experiment3      fork            input.txt       read            ssize_t
close.c    expl.program1.c exp1.program7    experiment3.c   fork.c          open            read.c          write
copy.txt   expl.program2    exp1.program8    experiment4      fork_exec       open.c          sample.c        write.c
dirname    expl.program3.c exp1.program8.c  experiment4.c   fork_exec.c     operating       sample.c.save
Child process completed.

```